MISAKA Network

耐量子ネイティブ Layer 1 ブロックチェーン
証明なし DAG コンセンサスを備えた仕様

MISAKA

MISAKA Network — コア開発
misakabtc@gmail.com
https://misakachain.com

ホワイトペーパー バージョン 1.0

2026年4月

概要

本文書は MISAKA Network の仕様を定義する。MISAKA Network は、署名および鍵交換のすべてを NIST 標準化済みの格子暗号のみで構築した Layer 1 ブロックチェーンである。ユーザから見えるすべての署名は ML-DSA-65(FIPS 204)署名であり、バリデーター間の全セッションは ML-KEM-768(FIPS 203)ハンドシェイクによって鍵付けされる。コンセンサスのクリティカルパスのいかなる箇所にも楕円曲線暗号プリミティブは存在しない。台帳は透明 UTXO モデルであり、未使用集合に対する SHA3-256 ベースの MuHash によるインクリメンタルコミットメントで認証される。コンセンサスは Mysticeti 参照実装を移植した Narwhal/Bullshark 型の証明なし DAG エンジンが提供し、良好なネットワーク条件下で 3 回のメッセージ遅延でブロックをコミットする。アクティブ委員会は、ステーク供給量の整数平方根重み付けによって選出される 21 名のスーパーリプレゼンタティブから成り、安全性はアクティブ総ステークの厳密に 3 分の 2 を超えるステーク加重ビザンチン定足数によって保証される。本 v1.0 リリースは、これまで個別に公開されていた Phase 1、Phase 2、Phase 3 のドラフトを、相互参照・自動生成目次・連続ページ番号を統一した単一文書として統合したものである。

目次

1. はじめに 6

1.1. 貢献 6

1.2. 文書の構成 6

2. システムモデルと前提 6

2.1. バリデーター、エポック、ステーク 6

2.2. 敵性モデル 7

2.3. 量子敵性者 7

2.4. ネットワークモデル 7

2.5. ステーク加重定足数 7

2.6. 目標 7

3. 暗号プリミティブ 8

3.1. ML-DSA-65(FIPS 204) 8

3.2. ML-KEM-768(FIPS 203) 8

3.3. ハッシュ関数 8

3.4. 記法 9

4. データモデル 9

4.1. プリミティブ型 9

4.2. アドレス 9

4.3. 出力と outpoint 9

4.4. ロックスクリプトとアンロック証拠 9

4.5. 入力とトランザクション 10

4.6. トランザクション識別子と sighash 10

4.7. UTXO 集合コミットメント(MuHash) 10

5. 認証子 11

6. トランザクションエフェクト 11

6.1. エフェクト構造 11

6.2. 決定性 11

6.3. エフェクトダイジェスト 11

6.4. 失敗したトランザクション 12

7. システム動作概観 12

7.1. コンポーネント 12

7.2. トランザクションのライフサイクル 12

8. バリデーターの永続状態 13

8.1. UTXO ストア 13

8.2. DAG ストア 13

8.3. コミット済みリーダーログ 13

8.4. バリデーター集合ストア 13

8.5. 供給量追跡 13

8.6. Write-ahead log 14

9. 権威の基本動作 14

9.1. トランザクションの受付 14

9.2. ブロックの受信 14

9.3. ブロックの提案 15

9.4. リーダーのコミットと実行 15

10. Narwhal/Bullshark コンセンサス 16

10.1. ブロック構造 16

10.2. ブロックの有効性規則 16

10.3. コミット規則 17

10.4. 線形化 17

10.5. ライブネスタイムアウト 17

11. エポック変更と再構成 17

11.1. エポック長 18

11.2. バリデーター選出 18

11.3. 投票重みと報酬重み 18

11.4. エポック遷移手続き 18

11.5. クロスエポック安全性 19

12. ライトクライアント 19

12.1. ヘッダーチェーン 19

12.2. 信頼アンカー: バリデーター集合 19

12.3. 定足数証明書の検証 19

12.4. 署名集約の不在 20

13. トークノミクス 20

13.1. 最大供給量と単位 20

13.2. 発行曲線 20

13.3. コインベース配分 20

13.4. 手数料モデル 21

13.5. ステーキングとボンディング 21

14. スケーリングと遅延 21

14.1. ブロックレベルのオーバーヘッド 21

14.2. ネットワークレベルのオーバーヘッド 21

14.3. 検証 CPU 21

14.4. 遅延 22

14.5. スループット 22

15. 安全性解析 22

15.1. 安全性 22
15.2. 耐量子性 23
15.3. 供給量のスナップショット永続性 23
16. 関連研究 23
17. 結論 24
参考文献 24
付録 A. スクリプトオペコード一覧 24
付録 B. 定数 25
付録 C. 謝辞 25

1. はじめに

現在本番稼動している公開ブロックチェーンの安全性は、楕円曲線離散対数問題の困難性に依存している。Bitcoin は secp256k1 を、Ethereum と EVM エコシステムは secp256k1 と BLS12-381 を、Solana・Sui・Move エコシステムは Ed25519 と BLS12-381 を、Cosmos は Ed25519 を、それぞれ採用している。Shor のアルゴリズム [1] は、十分に強力な量子コンピュータ上で同問題を多項式時間に還元するため、そのようなマシンが実現した瞬間に、これらのチェーンを守るすべての署名は原理的に偽造可能となる。オンチェーンに公開された公開鍵 ― UTXO の消費やトランザクション送信の時点で鍵は公開されるため、ほぼすべての鍵が該当する ― は、今日収集しておき将来破るという攻撃の対象である。

業界標準の対応は、ハードフォークを通じて将来的に耐量子暗号(PQC)へ移行するという計画である。しかしこの対応は構造的に不十分である。ハードフォークは新規の脆弱な署名の生成を止めることはできるが、既に公開された公開鍵を持つ出力を事後的に保護することはできず、敵対者がすでに記録してしまったネットワーク通信を守ることもできない。暗号学的に有意な量子コンピュータの登場後も維持される決済保証を提供することを目指すチェーンは、ジェネシスから耐量子である必要がある。

MISAKA Network はこの制約の下で設計された Layer 1 ブロックチェーンである。本システムは Rust で実装され、最大発行量を 10,000,000,000 MISAKA に固定した透明 UTXO 台帳を備え、Sui の Mysticeti 参照実装 [2] から移植した Narwhal/Bullshark 型 DAG コンセンサスエンジンによって保護されている。システム内のすべての署名 ― ユーザのトランザクション、バリデーターのブロック提案、DAG 投票、チェックポイント証明、バリデーター集合の更新証明書 ― は ML-DSA-65 署名 [3] である。バリデーター間のすべての TCP セッションは ML-KEM-768 ハンドシェイク [4] によって鍵付けされ、ChaCha20-Poly1305 で暗号化される。コンセンサスのクリティカルセクションには secp256k1、Ed25519、BLS、ペアリング系のいずれのコードパスも存在しない。この性質は、該当する依存関係の導入を拒否するビルド時リンターによって強制されている。

1.1. 貢献

本仕様は 4 つの貢献を行う。

(C1) 本文書は、コンセンサスエンジン・P2P トランスポート・ユーザレベルの署名方式のすべてを格子暗号のみで構築し、コンセンサスクリティカルパスから楕円曲線暗号を明示的に排除した Layer 1 ブロックチェーンの、完全な仕様を初めて提示する。

(C2) Mysticeti の証明なし DAG コミット規則 [2] を ML-DSA-65 で認証される UTXO 台帳向けに適合させ、その結果生じるオーバーヘッドを解析する。このオーバーヘッドは、プロトコル複雑性の増大ではなく、3309 バイトという ML-DSA-65 署名の固定サイズに支配されることを示す。

(C3) 発行量・流通量・ステーク量・焼却量を第一級の永続状態として記録する、スナップショット永続性のある供給量コミットメント構造(SupplyTracker)を定義する。さらに、バリデーター再起動時に供給量を再計算する実装に対しては可能であったリプレイベースの供給上限迂回攻撃のクラスが、本設計では排除されることを証明する(定理 7)。

(C4) 21 スロットのアクティブ委員会に対する、u128 上の決定的な分岐なし整数平方根を用いたステーク重み付け規則を定義する。本規則は、ステークに関する厳密な単調性と Sybil 耐性を保ったまま、線形重み付けに比して高ステークバリデーターの限界投票力を低減する。

1.2. 文書の構成

本文書は 3 部構成である。第 I 部(§2〜§6)ではプロトコルモデルを定義する。システムモデルと脅威モデル、暗号プリミティブ、台帳のデータ型、アドレスと認証子の形式、トランザクション実行が生成するエフェクトである。第 II 部(§7〜§11)は、バリデーター状態機械、番号付き手続きとしての権威オペレーション、Narwhal/Bullshark コンセンサスエンジン、DPoS 再構成プロトコルを規定する。第 III 部(§12〜§17)では、ライトクライアントプロトコル、トークノミクス、スケーリングと遅延の解析、形式化された定理を伴う安全性解析、関連研究、結論を扱う。さらに、スクリプトオペコード一覧、定数表、謝辞の 3 つの付録を付す。

2. システムモデルと前提

2.1. バリデーター、エポック、ステーク

MISAKA はエポックの系列として動作する。エポックは $e \in \mathbb{Z}$ で添字付けされ、ジェネシス時点で $e = 0$ である。各エポック e に対して次の組を定義する。

$$C_e = (V_e, S_e)$$

ここで V_e はエポック e にアクティブなバリデーター識別子の順序付き集合、$S_e : V_e \to \mathbb{Z}$ は各バリデーターの MISAKA

最小単位でのアクティブステークを返す関数である。C_e をエポック e の委員会と呼ぶ。委員会はエポックの期間中は固定で、エポック境界において §11 の再構成プロトコルによってのみ更新される。

MISAKA のアクティブ委員会サイズは全エポックで $|V_e| = 21$ に固定される。最小アクティブステークは MIN_STAKE_ACTIVE = 10^7 MISAKA (1 MISAKA = 10^9 最小単位であるから、等価的に 10^{16} 最小単位)である。ローテーション目的のため、10^6 MISAKA 以上をステークしている追加バリデーターから成るバックアップ集合がアクティブ委員会の外側で維持される。これらのバリデーターはコンセンサスには参加せず、§11 で規定される。

2.2. 敵性モデル

$T_e = \Sigma_{v \in V_e} S_e(v)$ をエポック e におけるアクティブ総ステークとする。計算能力に制限のある敵性者は、エポック e の開始時点において、任意のバリデーター部分集合 $B_e \subseteq V_e$ を、次の制約に従って静的に侵害できる。

$$\Sigma_{v \in B_e} S_e(v) < T_e / 3.$$

B_e に属するバリデーターをビザンチンと呼ぶ。これらはプロトコルから任意に逸脱し得る。たとえば、矛盾するメッセージへの署名、メッセージの保留、相互共謀などを行う可能性がある。残りの $V_e \setminus B_e$ に属するバリデーターは正直であり、プロトコルを厳密に遵守する。

これはステーク加重のビザンチン境界であり、個数加重ではないことに注意すべきである。条件 $|B_e| \lfloor 6$(n = 21 における個数加重類推値)は必要でも十分でもない。敵性者は、合計ステークが 3 分の 1 を超えているならばより少数のバリデーター侵害で済むこともあり、逆に合計ステークが少ないならばより多数のバリデーターを侵害できる。これは Sui [9] および Cosmos が採用するモデルと一致し、バリデーターの重みが不均一な任意のチェーンに対して適切なモデルである。

2.3. 量子敵性者

標準的な BFT モデルに対して敵性者を一方向に強化する。敵性者は暗号学的に有意な量子コンピュータ(CRQC)へのアクセスを持つと仮定する。具体的には、敵性者は楕円曲線離散対数問題、素因数分解問題、および古典的な離散対数系の任意の問題を期待多項式時間で解けるものとする。一方で、FIPS 203 および FIPS 204 が指定するパラメータ上の Module Learning-With-Errors(MLWE)問題および Module Short-Integer-Solution(MSIS)問題を効率的に解けるとは仮定しない。これらが MISAKA の安全性が依拠する唯一の計算量的困難性仮定である。

2.4. ネットワークモデル

正直なバリデーター間は、§3 の ML-KEM-768 ハンドシェイクによって確立された認証済みの点対点リンクで接続されている。ネットワークは Dwork, Lynch, Stockmeyer の意味で部分的に同期である。すなわち、未知の Global Stabilization Time(GST)と有限の遅延上界 Δ が存在し、正直なバリデーターが時刻 t に送信したメッセージは、時刻 max(GST, t) + Δ までに宛先に配送される。GST 以前は完全非同期であり、敵性者は任意にメッセージを遅延できる。安全性は非同期下でも成立すること、ライブネスは GST 以降に成立することを要求する。

2.5. ステーク加重定足数

本仕様全体を通じて、ステーク定足数(stake quorum)の概念を多用する。バリデーター集合 $Q \subseteq V_e$ がエポック e のステーク定足数であるとは、次を満たすときをいう。

$$\Sigma_{v \in Q} S_e(v) > (2 / 3) \cdot T_e.$$

実装では浮動小数点演算を避けるため、この値を整数演算で $\lfloor (2 \cdot T_e / 3) \rfloor + 1$ に切り上げる。定足数交叉(quorum intersection)の標準的な議論により、任意の 2 つのステーク定足数は、合計ステークが $T_e / 3$ を超える集合で交わる。よって §2.2 の境界の下で、両者は少なくとも 1 名の正直なバリデーターを共有する。この交叉補題は、§15 の安全性解析で最も頻繁に用いられる証明の道具である。

個数加重定足数に関する注記。 Mysticeti 仕様 [2] は、均一ステークの n = 3f + 1 サイズの委員会と 2f + 1 個のバリデーターによる定足数を前提に記述されている。MISAKA は全体を通してそのステーク加重一般化を用いる。本文書で Mysticeti の補題を引用する場合は、その補題のステーク加重版を指している。証明は定足数交叉のみを使用するため、ステーク加重定式化でも修正なしに成立する。

2.6. 目標

MISAKA はビザンチン原子ブロードキャスト(BAB)プロトコル [5], [6] である。次の 4 つの標準的性質を要求する。

合意(Agreement)。 正直なバリデーター v_i がトランザクション tx を配送したならば、すべての正直なバリデーター v_j もいずれ tx を配送する。

整合性(Integrity)。 各正直なバリデーターは各トランザクションを高々

1 回配送し、配送するのはいずれかのバリデーターが事前にブロードキャストした場合に限る。

有効性(Validity)。 正直なバリデーターがトランザクションをブロードキャストしたならば、すべての正直なバリデーターがいずれそれを配送する。

全順序(Total order)。 2 名の正直なバリデーター v_i と v_j がトランザクション tx と tx′ を配送するならば、両者は同一の相対順序でそれらを配送する。

加えて耐量子前方秘匿性を要求する。本日記録されたネットワーク通信は、敵性者が将来 CRQC を獲得しても機密性を維持し続けなければならない。

3. 暗号プリミティブ

MISAKA はコンセンサスおよび台帳コードにおいて 3 つの暗号プリミティブを使用する。デジタル署名用の ML-DSA-65、鍵カプセル化用の ML-KEM-768、衝突耐性ハッシュ用の SHA3-256 である。加えて BLAKE3 が非コンセンサス経路(mempool 重複排除、write-ahead-log のチェックサム)において、標準化より処理速度が重要な箇所で使用される。本節では各プリミティブとそれが課す安全性仮定を定義する。

3.1. ML-DSA-65(FIPS 204)

ML-DSA は、CRYSTALS-Dilithium 投稿に基づき NIST が FIPS 204 [3] として標準化した Module-Lattice-based Digital Signature Algorithm である。MISAKA はパラメータセット ML-DSA-65 を使用する。これは NIST 安全性カテゴリ3 に対応し、古典約 192 bit、耐量子約 128 bit の安全性を提供する。本方式は次の 3 つのアルゴリズムから成る。

```
KeyGen() → (pk, sk)
Sign(sk, m) → σ
Verify(pk, m, σ) → {0, 1}
```

具体的なサイズは |pk| = 1952 バイト、|sk| = 4032 バイト、|σ| = 3309 バイトである。本方式は FIPS 204 のパラメータセット上の MLWE および MSIS 仮定の下で、多項式時間の量子敵性者に対して適応的選択メッセージ攻撃下での存在的偽造不可能性(EUF-CMA)を持つと仮定する。§15 の認証関連性質に関する証明はすべてこの仮定に帰着される。

ML-DSA-65 は次の用途に未改変のまま使用される。(i) 各 UTXO 入力のアンロック署名(§4)、(ii) 各コンセンサスブロックの作成者署名(§10)、(iii) バリデーター間ハンドシェイクにおけるオペレーター鍵認証、(iv) エポック境界におけるチェックポイント証明。

3.2. ML-KEM-768(FIPS 203)

ML-KEM は、CRYSTALS-Kyber 投稿に基づき NIST が FIPS 203 [4] として標準化した Module-Lattice-based Key Encapsulation Mechanism である。MISAKA はパラメータセット ML-KEM-768 を使用する。これは NIST 安全性カテゴリ 3 に対応する。本方式は次の 3 つのアルゴリズムから成る。

```
KeyGen() → (pk, sk)
Encaps(pk) → (ct, K)
Decaps(sk, ct) → K
```

ここで K は 32 バイトの共有秘密である。本方式は MLWE 仮定の下で多項式時間の量子敵性者に対して IND-CCA2 安全性を持つと仮定する。

MISAKA はバリデーター間のすべての TCP セッションを、ML-KEM-768 カプセル化を実行し、得られた秘密から HKDF-SHA3-256 で AEAD 鍵ペアを導出し、両端点を ML-DSA-65 署名でハンドシェイク transcript にわたって認証することで確立する。得られたセッションは ChaCha20-Poly1305 で暗号化される。X25519+ML-KEM のハイブリッド構成は意図的に採用しない。ハンドシェイクに楕円曲線成分が含まれると、CRQC が現実化した後にハーベストナウ・デクリプトレイター攻撃に対して脆弱となり、耐量子設計の目的を根本から損なうためである。セッション寿命は設定可能なパラメータ MAX_SESSION_LIFETIME_SECS によって上限を持つ。テストネット既定値は 86400 秒、バリデーターに対する推奨値は 14400 秒で、それを超えるとハンドシェイクを再実行する。

3.3. ハッシュ関数

MISAKA は 2 つのハッシュ関数を、それぞれ明確に定められた範囲で使用する。

SHA3-256 [7] はコンセンサスから可視のハッシュ関数である。トランザクション識別子、ブロックハッシュ、マークル木枝、MuHash UTXO アキュムレータ(§4.7)、アドレスチェックサム、セッション鍵導出の HKDF、および ML-DSA-65 署名で署名されるあらゆる値の計算に使用される。SHA3-256 は、標準的な Grover 速度向上を考慮した多項式時間の量子敵性者に対して衝突耐性、原像耐性、第二原像耐性を持つと仮定する。

BLAKE3 は非コンセンサス経路のみで使用される。mempool の重複排除、write-ahead-log のブロックチェックサム、内部キャッシュキー等である。BLAKE3 でハッシュ化された値がコンセンサス状態に格納されたり、署名され

たり、ブロックから参照されたりすることはない。動機はスループットである。BLAKE3 は一般的なハードウェア上で SHA3-256 よりおよそ一桁高速であり、非コンセンサス経路はその恩恵を受ける一方、安全性のために BLAKE3 への強い依存を付け加えることがない。耐量子性を強制するビルド時リンターはこの範囲境界もあわせて強制する。

3.4. 記法

SHA3-256 を H(·) と書く。ML-DSA-65 の演算は $\mathrm{Sign}_{sk}(m)$ および $\mathrm{Verify}_{pk}(m, \sigma)$ と書く。文脈から明らかな場合はアルゴリズム名を省略する。バイト列の連結は ‖ で表記する。〈·〉 記法は、H への入力として使用される決定的なバイトレベルエンコーディング(§4.6 で規定)を表す。

4. データモデル

MISAKA は透明な UTXO 台帳である。任意の時点での台帳状態は、未使用トランザクション出力の集合と、少量の補助状態(供給量追跡、バリデーター集合、エポック番号)によって完全に記述される。本節ではオンチェーン型を、Rust 構造体定義またはワイヤーフォーマットスキーマへ直接翻訳可能な形式で定義する。

4.1. プリミティブ型

表 1 にデータモデルの他の部分が使用するプリミティブ型を挙げる。

型	定義
U64	64 ビット符号なし整数、ワイヤー上はリトルエンディアン。
U128	128 ビット符号なし整数、ステークおよび供給量に使用。
Bytes32	固定長 32 バイト文字列。
ChainID	U32、1 = メインネット、2 = テストネット、その他予約。
Round	U64、単調なラウンド論理番号、0 から開始。
Epoch	U64、単調なエポック番号、0 から開始。
Hash	Bytes32、H(·) = SHA3-256 の出力。
MLDSAPubKey	1952 バイトの ML-DSA-65 公開鍵。
MLDSASig	3309 バイトの ML-DSA-65 署名。

表 1: プリミティブ型。

4.2. アドレス

MISAKA におけるアドレスは、ML-DSA-65 公開鍵の SHA3-256 ハッシュを、チェーンに紐づけられたテキスト包装に入れたものである。形式的には次のとおりである。

フィールド	型	説明
raw	Bytes32	H("MISAKA:addr:v2:" ‖ pk)。pk は ML-DSA-65 公開鍵。
chain_id	ChainID	このアドレスが紐づいているチェーン識別子。
checksum	U16	H("MISAKA:addr:checksum:v2:" ‖ chain_id_le4 ‖ hex(raw)) の先頭 2 バイト。

表 2: アドレス構造。

テキストエンコーディングは misaka1 ‖ hex(raw) ‖ hex(checksum) である。chain_id がチェックサムの原像に含まれるため、同一の raw バイト列でも、テストネット(chain_id = 2)とメインネット(chain_id = 1)では構文上別個のテキストアドレスが生成される。これにより、クロスチェーンアドレス混同攻撃のクラス全体が検証層で排除される。

4.3. 出力と outpoint

フィールド	型	説明
value	U64	最小単位での数量(1 MISAKA = 10^9 最小単位)。
lock	LockScript	使用条件。§4.4 を参照。

表 3: TxOut 構造。

outpoint は、特定のトランザクションが生成した特定の出力を識別する。これは組 (txid, index) であり、txid は出力を生成したトランザクションのハッシュ、index はそのトランザクションの出力リスト内における 0 始まりの位置である。

4.4. ロックスクリプトとアンロック証拠

すべての出力はロックスクリプトを持つ。これは MISAKA スクリプト言語で書かれた短いプログラムであり、その出力の使用条件を定義する。スクリプト言語は、Bitcoin Script および Kaspa Script のスタック機械方式であり、オペコードの小集合に制限され、耐量子署名チェック命令で拡張されている。2 つの主要なオペコードは次のとおりである。

OP_CHECKSIG_MLDSA: スタックから 1952 バイトの公開鍵と 3309 バイトの署名をポップし、§4.6 で定義されるトランザクション sighash z を計算し、$\mathrm{Verify}_{pk}(z, \sigma) = 1$ なら 1 を、そうでなければ 0 をプッシュする。

OP_CHECKSIGVERIFY_MLDSA:
OP_CHECKSIG_MLDSA と同一であるが、検証が 0 を返した場合即座にスクリプトを失敗させる。

secp256k1 または Ed25519 を呼ぶレガシーの OP_CHECKSIG オペコードは MISAKA スクリプト集合には存在せず、そのようなオペコードを含むスクリプトをデシリアライズしようとする試みは、コンセンサス層において決定的なエラーコードで拒否される。v1 の完全なオペコード集合は付録 A に示す。

4.5. 入力とトランザクション

フィールド	型	説明
outpoint	(Hash, U32)	この入力が消費する (txid, index) の組。
unlock	UnlockWitness	消費される出力のロックスクリプトを満たす証拠。
sequence	U32	入力ごとのシーケンス番号。将来の用途のために予約されており、v1 では 0xFFFFFFFF でなければならない。

表 4: TxIn 構造。

フィールド	型	説明
version	U16	トランザクション形式バージョン。現在は 1。
chain_id	ChainID	このトランザクションが有効なチェーン。
inputs	Vec<TxIn>	非コインベースでは非空、コインベースでは空。
outputs	Vec<TxOut>	少なくとも 1 つの出力。
lock_time	U64	このトランザクションが包含可能となる最小ラウンド番号。ロックなしの場合は 0。
fee	U64	最小単位の明示的な手数料。sum(inputs) − sum(outputs) に等しくなければならない。

表 5: トランザクション構造。

トランザクションがwell-formedであるとは、そのシリアライゼーションが正準(末尾バイトなし、整数エンコーディングが固定、フィールドが辞書順)であり、すべての出力値が U64 に収まり、明示的な fee フィールドが入出力差分に一致し、chain_id がバリデーターのローカルチェーン識別子と一致することをいう。well-formed なトランザクションが台帳状態 Σ に対して有効であるとは、入力 outpoint がすべて Σ に存在し、各入力のアンロック証拠が対応するロックスクリプトを下記 sighash の下で満たし、入力値の和が出力値の和と明示的手数料の合計以上であることをいう。

4.6. トランザクション識別子と sighash

トランザクション識別子は次で定義される。

$$\mathrm{txid} = H(\texttt{"MISAKA:txid:v1:"} \parallel \langle \mathrm{tx_without_witnesses} \rangle)$$

ここで 〈tx_without_witnesses〉 は、各 unlock フィールドを長さ 0 のプレースホルダーで置き換えたトランザクションの正準シリアライゼーションである。これにより txid は証拠データに依存しないため、第三者によるトランザクション可鍛性を防止する上で本質的である。

トランザクション tx の入力 i に対するsighash は次で定義される。

$$\mathrm{sighash}_i = H(\texttt{"MISAKA:sighash:v1:"} \parallel \mathrm{chain_id} \parallel \mathrm{txid} \parallel i \parallel \mathrm{lock_script}_i),$$

ここで $\mathrm{lock_script}_i$ は入力 i が消費する出力のロックスクリプトである。ロックスクリプトを含めることで、署名が使用対象の特定出力に紐づき、アドレスを共有する複数の出力にまたがる証拠の使い回しを防止する。チェーン識別子を含めることで、アドレス形式とは独立にクロスチェーントランザクションリプレイを防止する。

4.7. UTXO 集合コミットメント(MuHash)

バリデーターが保持する未使用トランザクション出力集合のビューは、MuHash と呼ばれるインクリメンタル多重集合ハッシュによって要約される。MISAKA の実装は misaka-muhash で定義される SHA3-256 ベースの変種を用いる。MuHash は、バイト列の多重集合をそれらのハッシュの大きな有限群における積として表現する。要素の挿入および削除はその群上の乗算および除算に対応し、いずれも定数時間である。したがって、ブロック適用後の UTXO 集合の状態コミットメントは、そのブロックが触れる入力および出力の個数に比例する時間で計算でき、UTXO 集合全体のサイズに比例する必要はない。

具体的には、各 UTXO は 〈outpoint ‖ value ‖ lock_script〉 として正準に符号化され、SHA3-256 でハッシュされ、バリデーターの MuHash 状態にアキュムレートされる。この状態の 32

バイトダイジェストは、state_root というフィールド名で各ブロックに格納される。同一のジェネシス状態から開始して同一のコミット済みトランザクション系列を適用した 2 名の正直なバリデーターは、MuHash が可換であるため、実行順序によらずビット単位で同一の state_root 値を生成する。ローカルで計算した state_root がコミット値と乖離した任意のバリデーターは停止してフォーク警告を発する。これは無条件に強制されており、独立実行検証が行われる仕組みである。

5. 認証子

§4.2 でアドレスの構造を定義した。本節では認証子を定義する。これはアンロック証拠が持つデータであり、トランザクションがアドレスに対応する秘密鍵の保持者によって権限付けされたことを証明する。

標準の単一署名ロックに対する認証子は次の組である。

```
Auth = (pk : MLDSAPubKey, σ : MLDSASig)
```

この認証子は、次の両方が成り立つとき、かつそのときに限り、入力 i に対する OP_CHECKSIG_MLDSA スクリプトによって受理される。

(A1) pk から H("MISAKA:addr:v2:" ‖ pk) によって導出されるアドレスが、満たされるべきロックスクリプトにコミットされたアドレスに等しい。

(A2) $\mathrm{Verify}_{pk}(\mathrm{sighash}_i, \sigma) = 1$。ここで $\mathrm{sighash}_i$ は §4.6 で定義された値。

複数署名ロックおよび閾値ロックは v2 スクリプト拡張のため予約されており、本 v1 仕様には含まれない。想定される符号化は ML-DSA-65 鍵上の MuSig 型の集約であるが、本ドラフト時点で ML-DSA に対する本番運用可能な集約方式は存在しないため、そのような方式が利用可能になるまで仕様策定を先送りする。

6. トランザクションエフェクト

トランザクションが台帳状態に対して実行されると、そのエフェクトと呼ばれる構造化値が生成される。エフェクトは、トランザクションが行ったことを正確に記録し、任意の独立した実行器によって再現可能な形式にしたものである。これは state_root が計算される単位であり、ライトクライアント(§12)が検証する単位でもある。

6.1. エフェクト構造

フィールド	型	説明
txid	Hash	これらのエフェクトを生成したトランザクションの識別子。
status	Status	{Success, Failure(code)} のいずれか。Success のみ状態変更をコミットする。
consumed	Vec<OutPoint>	UTXO 集合から削除された outpoint。
created	Vec<(OutPoint, TxOut)>	UTXO 集合に追加された新規出力と、割り当てられた outpoint。
fee_paid	U64	コミットブロックの手数料プールに移転された最小単位の金額。
epoch	Epoch	トランザクションがコミットされたエポック。
round	Round	トランザクションを含むブロックがコミットされたラウンド。

表 6: トランザクションエフェクト。

6.2. 決定性

エフェクトの計算は決定的であることが要求される。同一の開始状態に対して同一のトランザクションを実行する 2 名の正直なバリデーターは、ビット単位で同一のエフェクトを生成しなければならない。これは浮動小数点演算、ハッシュマップ上の非決定的な走査、システム時刻の読み取りなど、プラットフォーム依存の振る舞いをもたらすあらゆる要素を排除する。MISAKA 参照実装は 3 つの仕組みで強制する。(i) 金額演算はすべて浮動小数点ではなくチェック付き u128 演算を使用する。(ii) コレクションの走査はすべて正準順序(キーの辞書順)で行う。(iii) ステーク重み付けで用いる整数平方根は、libm 呼び出しではなく分岐なし二分探索として実装する。

6.3. エフェクトダイジェスト

エフェクトダイジェストは次で定義される。

```
digest = H("MISAKA:effect:v1:" ‖ <effect>).
```

エフェクトダイジェストはブロック生成者によって集約され、各ブロックの effects_root フィールドに入る。これは当該ブロックで実行されたすべてのトランザクションのダイジェストの SHA3-256 マークルルートとして計算される。ライトクライアント(§12)は個々のトランザクションエフェクトを、完全なブロックペイロードをダウンロードすることなく、このルートに対するマークル包含証明を検証することで確認する。

6.4. 失敗したトランザクション

スクリプト検証に失敗したトランザクション、入力値が出力値と手数料の合計を下回るトランザクション、および lock_time に達していないトランザクションは、状態 Failure(code)
でコミットされる。失敗したトランザクションは UTXO を消費せず、UTXO を生成せず、手数料も支払わないが、依然としてブロック順序上の位置を占め、エフェクトレコードを生成する。これは Sui [9]
が採用している慣例と同一であり、2 名のバリデーターがトランザクションの試行有無について合意するために必要である。たとえ別の順序の下でならば成功していたかどうかについて両者の見解が異なっていたとしても、この合意は維持される。

7. システム動作概観

本仕様の第 I 部(§2〜§6)ではデータモデルを定義した。すなわち、トランザクションとは何か、アドレスとは何か、エフェクトとは何かである。第 II 部(§7〜§11)では、バリデーターがこれらのオブジェクトに対して何を行うかを定義する。本節ではバリデーター状態機械とその主要な相互作用の高水準の概観を示す。詳細は §8〜§11 で規定する。

7.1. コンポーネント

各 MISAKA バリデーターは 5 つの論理コンポーネントを動作させる。参照実装ではこれらは別個の Rust クレートとして実装されているが、単一のプロセス内アドレス空間を介して接続されている。

(1) P2P トランスポート(misaka-p2p)は、他のすべてのバリデーターに対する認証済み暗号化 TCP セッションを維持する。セッションは ML-KEM-768 ハンドシェイク(§3.2)によって確立され、ChaCha20-Poly1305 でレコード暗号化する。

(2) メンプール(misaka-mempool)は、well-formed だがまだ包含されていないトランザクションをバッファする。手数料率順序、トランザクション識別子による重複排除、送信者ごとの受付ポリシーを強制する。

(3) コンセンサスエンジン(misaka-dag)は、Mysticeti の証明なし DAG プロトコル [2]
の移植である。署名付きブロックを生成し、P2P トランスポートを介してゴシップし、結果として得られる DAG を解釈してどの提案者スロットがコミットされるかを決定し、コミットされたリーダーブロックの全順序系列を実行層に出力する。§10 で詳述する。

(4) 実行エンジン(misaka-execution)は、コミットされたリーダーブロック系列を消費し、含まれるトランザクションを決定的順序で UTXO
集合に適用し、MuHash 状態コミットメントを更新する。台帳状態を変更するのはこのコンポーネントのみである。

(5) ストレージ層(misaka-storage, misaka-database)は、UTXO 集合、供給量追跡、バリデーター集合、コンセンサス
DAG、クラッシュリカバリ用の write-ahead-log を永続化する。永続化される唯一のコンポーネントであり、他のすべてはこれから再構築可能である。

7.2. トランザクションのライフサイクル

トランザクションがクライアント送信から最終コミットまで MISAKA 内を流れる経路は、次の 7 段階から成る。

(L1) クライアントは §4 のデータモデルを用いてトランザクションを構築し、各入力を ML-DSA-65 秘密鍵で該当する sighash に対して署名し、1 つ以上のバリデーター RPC
エンドポイントに送信する。

(L2) 受信したバリデーターは、§4.5 の well-formed 性チェックを実行し、消費される各 outpoint をローカル UTXO 集合から検索し、各アンロック証拠を対応するロックスクリプトに対して検証し、すべてのチェックが通ればメンプールに受け入れる。

(L3) バリデーターが自身のブロック提案の番になると、手数料率順に並べたトランザクションの一群をメンプールから取り出し、新しいブロックのペイロードに含め、ブロックを ML-DSA-65
オペレーター鍵で署名し、P2P トランスポート経由で他のすべてのバリデーターにブロードキャストする。

(L4) 他のバリデーターはブロックを受信し、作成者署名を検証し、ブロックの親参照が §10.2 の有効性規則を満たしていることを確認し、ブロックをローカル DAG に格納し、自分が次ラウンドで提案するときにそのブロックを自分のブロックの親として参照する。

(L5) さらにラウンドが経過すると、§10.3 の Mysticeti コミット規則は最終的にそのブロックを含むリーダースロットを to-commit または to-skip のいずれかに分類する。to-commit の場合、リーダーの因果履歴が決定的なトランザクション順序に線形化され、実行エンジンに送出される。

(L6) 実行エンジンは各トランザクションを順に適用し、MuHash 状態コミットメントをインクリメンタルに再計算し、トランザクションごとのエフェクトレコード(§6)を生成する。結果の state_root をコミット済みブロックが保持する値と照合する。不一致の場合、バリデーターは停止しフォーク警告を発する。

(L7) この時点でトランザクションは最終的である。エフェクトは永続化され、消費された UTXO はストレージ層から削除され、新しい UTXO は後続のトランザクションから使用可能となり、トランザクションのエフェクトダイジェストはブロックの effects_root からライトクライアント検証可能となる。

L1〜L4 にかかる時間はネットワークの往復時間で制限される。L5〜L7 はコンセンサスのコミット遅延を要し、良好なネットワーク条件下ではさらに 3 ラウンドである。

8. バリデーターの永続状態

各 MISAKA バリデーターは小さな永続ストア集合を保持する。本節では各ストア、そのキー型・値型、および整合性要件を定義する。命名規則は Sui [9, §4.2] に倣う。X_v という名前のストアは、バリデーター v が保持する X のローカルコピーを表す。

8.1. UTXO ストア

ストア名: $UTXO_v$。キー型: OutPoint = (Hash, U32)。値型: TxOut = (U64, LockScript)。意味: 現在未使用のすべての出力を、その値とロックスクリプトにマップする。

$UTXO_v$ は、与えられたトランザクションの有効性を判定するために必要かつ十分な唯一のストアである。トランザクションが $UTXO_v$ に対して有効であるのは、消費されるすべての outpoint がストアに存在し、かつすべてのアンロック証拠が検証を通るとき、かつそのときに限る。本ストアは MuHash アキュムレータと同期して原子的に更新される。各挿入はアキュムレータの乗算と対を成し、各削除は除算と対を成す。両者は同一のストア書き込みトランザクション内でコミットされ、ストアとアキュムレータが食い違う瞬間は存在しない。

8.2. DAG ストア

ストア名: DAG_v。キー型: BlockID = Hash。値型: Block(§10.1 で定義)。意味: バリデーター v が受信し well-formed と受理したすべてのブロックを、内容ハッシュで添字付けして保持する。

DAG ストアは単一エポック内では追記のみである。ブロックは追加されるが削除や変更はされない。エポック境界では再構成プロトコル(§11)により、後続の state-sync 要求に必要なデータのみを残すよう刈り込まれる。これは retention_rounds パラメータ(既定 10000)によって制御される。

8.3. コミット済みリーダーログ

ストア名: L_v。キー型: U64(コミット index)。値型: BlockID。意味: v のコンセンサスエンジンがコミットを決定したリーダーブロック識別子の全順序系列であり、グローバルコミット順における位置で添字付けされている。

ビザンチン原子ブロードキャストの合意性(§2.6)は、任意の2 名の正直なバリデーター v_i と v_j について、L_{v_i} の内容が L_{v_j} の接頭辞であるか、逆が成り立つことを主張するものである。これが §10 のコンセンサスエンジンが保証しなければならない唯一の性質である。

8.4. バリデーター集合ストア

ストア名: VS_v。キー型: Epoch。値型: Committee = (Vec<ValidatorID>, Vec<U128>, Vec<MLDSAPubKey>)。意味: 各エポックに対する、アクティブバリデーター識別子の順序付きリスト、それぞれの最小単位アクティブステーク、およびオペレーター公開鍵。

バリデーター集合ストアはエポックごとに 1 回、再構成プロトコル(§11)によって更新される。ステーク加重定足数計算の唯一の真実の源である。コンセンサスエンジンが行うすべての定足数チェックは、現在のエポック e について $VS_v[e]$ から読み出す。

8.5. 供給量追跡

ストア名: $Supply_v$。シングルトン(キーなし)。値型: SupplyTracker = (total: U128, circulating: U128, staked: U128, burned: U128, max: U128)。意味: チェーンの現在の金銭状態。

各コインベーストランザクションは total と circulating を新規発行量だけ加算する。各ステーク操作は circulating と staked の間で値を移動する。各手数料焼却(もしあれば)は burned を加算する。すべての更新はチェック付き U128 演算を用いて行われ、該当する UTXO 書き込みと同一のストアトランザクション内でコミットされる。制約 total ≤ max はすべての発行で強制され、これを違反する発行の試みは SupplyError::ExceedsCap を返し、当該ブロックは拒否される。

スナップショット永続性。 $Supply_v$ はチェックポイント境界のみならず、すべてのコミットで永続化される。起動時にこのストアを読み込めないバリデーターは、ゼロ初期化された追跡でフォールバックする代わりに起動を拒否する。これにより、再起動されたバリデーターが既にコミット済みの報酬を再発行することが阻止される。これは §1.1 の貢献 C3 であり、§15 の定理 7 によって形式的に証明される。

8.6. Write-ahead log

ストア名: WAL_v。追記専用レコード系列。各レコードは単調なシーケンス番号、レコード型タグ、レコードペイロード、BLAKE3 チェックサムを保持する。WAL はコンセンサスのクリティカルパス上で書き込まれる唯一のストアであり、他のストアは起動時のリカバリ掃引中に WAL レコードから遅延的に更新される。これは Mysticeti 参照設計 [2, §VI] に従うもので、コンセンサスエンジンがブロックを単一のベクトル化書き込みでコミットできるようにし、複数ストアトランザクションを避ける。

9. 権威の基本動作

MISAKA における権威(authority)とは、単独で考察される 1 名の正直なバリデーターを指す。その役割は、受信メッセージに応答してローカル状態に対する明確に定義された操作の小集合を適用することであり、すべての正直な権威のグローバルな相互作用がビザンチン原子ブロードキャストを提供するように設計されている。本節ではその操作を番号付き手続きとして規定する。

以下の記法を全体で使用する。v はローカル権威、e は現在のエポック。$UTXO_v$、DAG_v、L_v、VS_v、$Supply_v$ は §8 の永続ストア。$S_e(\cdot)$ と T_e は §2.1 のバリデーターごとおよびアクティブ総ステークである。

9.1. トランザクションの受付

ローカル RPC インターフェースまたはゴシップ層が権威にトランザクション tx を渡すと、以下の手続きによってメンプールへの受け入れ可否を判定する。手続きは全域的で、すべての入力は成功するか決定的なエラーコードを返す。

```
手続き 1: process_transaction(tx)
 1. if not well_formed(tx): return Err(MALFORMED)
 2. if tx.chain_id ≠ self.chain_id: return
 Err(CHAIN_ID)
 3. for each input i in tx.inputs:
     a. let utxo := UTXO_v[i.outpoint]
     b. if utxo = ⊥: return Err(MISSING_INPUT)
     c. let z := sighash(tx, i, utxo.lock)
     d. if not script_eval(utxo.lock, i.unlock,
 z):
          return Err(BAD_WITNESS)
 4. let in_sum := Σ UTXO_v[i.outpoint].value for i
 in tx.inputs
 5. let out_sum := Σ o.value for o in tx.outputs
 6. if in_sum < out_sum + tx.fee: return
 Err(INSUFFICIENT)
 7. if mempool.contains(txid(tx)): return
 Ok(DUPLICATE)
 8. mempool.insert(tx, fee_rate := tx.fee /
 size(tx))
 9. gossip.broadcast(tx)
 10. return Ok(ADMITTED)
```

process_transaction は $UTXO_v$ に対して読み取り専用であり、入力を消費しない。なぜならトランザクションはまだコンセンサスによって順序付けされておらず、別の提案者のブロックにある別のトランザクションと競合する可能性があるためである。実際の消費は下記の execute_block で行われる。

9.2. ブロックの受信

ゴシップ層が署名付きブロック b を配送すると、権威はそれを検証しローカル DAG に挿入する。この手続きは §10.2 の構造規則に違反する任意のブロックを拒否するが、そのブロックがコミットされるかの決定はまだ行わない。その決定は下記の try_commit が行う。

手続き 2: process_block(b)

```
1. if not well_formed_block(b): return
Err(MALFORMED)
2. if b.epoch ≠ self.epoch: return
Err(WRONG_EPOCH)
3. let pk := VS_v[b.epoch].pk_of(b.author)
4. if pk = ⊥: return Err(UNKNOWN_AUTHOR)
5. if not Verify_pk(block_sighash(b),
b.signature):
    return Err(BAD_SIGNATURE)
6. for each parent p in b.parents:
    a. if DAG_v[p] = ⊥: request p; suspend(b)
7. if b.parents.first ≠
latest_self_parent(b.author):
    return Err(BAD_SELF_PARENT)
8. if not stake_quorum(b.parents.authors at round
b.round-1):
    return Err(INSUFFICIENT_PARENTS)
9. DAG_v.insert(block_id(b), b)
10. WAL_v.append(BlockReceived { id :=
block_id(b) })
11. try_commit()
12. return Ok(ACCEPTED)
```

ステップ 6(request and suspend)は標準的な DAG 同期機構である。親がローカルに未到着のブロックは待機キューに保留され、欠けている親が到着した時点で再評価される。ステップ 7 は §10.2 B3 の first-self-parent 規則を強制する。これはフォークをまたいで暗黙の認証構造を一貫させるために必要である。ステップ 8 は Mysticeti の「少なくとも 2f + 1 個の親」規則のステーク加重類推であり、§2.5 の定足数述語を用いる。

9.3. ブロックの提案

権威がラウンド r − 1 のブロックの定足数を受理した時点で、自らラウンド r のブロックを構築してブロードキャストする。

手続き 3: propose_block(r)

```
1. let parents := select_parents(r) # see §10.2
2. let payload :=
mempool.drain_top(MAX_BLOCK_BYTES)
3. let b := Block {
    author := self.id,
    round := r,
    epoch := self.epoch,
    parents := parents,
    payload := payload,
    timestamp := now(),
    state_root := UTXO_v.muhash(),
    effects_root:= last_committed_effects_root,
    signature := ⊥,
  }
4. b.signature := Sign_sk(block_sighash(b))
5. DAG_v.insert(block_id(b), b)
6. WAL_v.append(BlockProduced { id := block_id(b)
})
7. gossip.broadcast(b)
```

ステップ 4 の署名演算は、コンセンサススレッドがブロックしないよう、本番実装では専用のワーカースレッドに委譲される。ここに示した抽象手続きは明瞭さのため逐次的である。state_root フィールドは、提案中のブロック時点ではなく、最後にコミットされたリーダー時点の UTXO 集合の MuHash を保持する。後者では、提案者が並行する提案の線形化順序を予測する必要があり、先にコミットせずには不可能であるためである。

9.4. リーダーのコミットと実行

process_block の成功後ごとに、権威は try_commit を呼び出す。これは §10.3 の Mysticeti 決定規則を適用し、コミット済みリーダーログ L_v を前進させようと試みる。

手続き 4: try_commit()

```
1. let new_commits := mysticeti_decide(DAG_v,
L_v)
2. for each leader_id in new_commits:
    a. let leader := DAG_v[leader_id]
    b. let txs :=
linearize(causal_history(leader))
    c. execute_block(leader, txs)
    d. L_v.append(leader_id)
    e. WAL_v.append(LeaderCommitted {
        id := leader_id,
        state_root := UTXO_v.muhash(),
    })
```

linearize は §10.4 で規定されるリーダーの因果履歴の決定的後行順走査である。execute_block は各トランザクションを順に適用する。

```
手続き 5: execute_block(leader, txs)
 1. let fee_pool := 0
 2. for each tx in txs:
     a. if revalidate(tx) = Err(_):
         effects.append(Failure(tx))
         continue
     b. for each i in tx.inputs:
         UTXO_v.remove(i.outpoint)
     c. for each (j, o) in enumerate(tx.outputs):
         UTXO_v.insert((txid(tx), j), o)
     d. fee_pool += tx.fee
     e. effects.append(Success(tx))
 3. let cb := build_coinbase(leader, fee_pool,
 Supply_v)
 4. for each (j, o) in enumerate(cb.outputs):
     UTXO_v.insert((txid(cb), j), o)
 5. Supply_v.mint(cb.minted)? # may return
 ExceedsCap
 6. let local_root := UTXO_v.muhash()
 7. if local_root ≠ leader.state_root:
     halt(); raise ForkAlert(local_root,
 leader.state_root)
 8. WAL_v.append(BlockExecuted {
     leader := leader.id,
     effects_root := merkle(effects),
   })
```

ステップ 2a(revalidate)は必要である。なぜなら手続き 1 はトランザクションをそれより早い UTXO 状態に対して受け入れたのであり、トランザクションが実行される時点では、同じ線形化順序中の先行トランザクションがすでにその入力の 1 つを消費している可能性があり、その場合 §6.4 に従って Failure としてコミットされるためである。ステップ 7 は、ビザンチンな提案者が任意の事後状態をコミットするのを阻む独立状態ルート検証である。

10. Narwhal/Bullshark コンセンサス

MISAKA のコンセンサスエンジンは、Mysticeti の証明なし DAG プロトコル [2] を UTXO 台帳および ML-DSA-65 署名に移植したものである。本節ではプロトコルの操作的仕様を与え、安全性解析は §15 に送る。本文書が自己完結するよう Mysticeti のコミット規則を必要十分再掲するが、完全な証明については [2, §V] を参照する。

10.1. ブロック構造

フィールド	型	説明
author	ValidatorID	提案バリデーターの V_e における添字。
round	Round	単調増加する論理ラウンド番号。
epoch	Epoch	ブロックが有効なエポック。
parents	Vec<BlockID>	親ブロックへの参照。先頭は作成者自身の前ブロック。
payload	Vec<Tx>	包含される順序付きトランザクションリスト。
timestamp	U64	提案時の壁時計時刻。作成者ごとに単調。
state_root	Hash	最後にコミットされたリーダー時点の UTXO 集合の MuHash。
effects_root	Hash	最後にコミットされたリーダーのエフェクトダイジェストのマークルルート。
signature	MLDSASig	先行するすべてのフィールドに対する作成者の ML-DSA-65 署名。

表 7: ブロック構造。

10.2. ブロックの有効性規則

ブロック b がバリデーター v のローカル DAG において有効であるとは、次のすべてが成り立つときをいう。

(B1) 署名が $VS_v[b.epoch]$ に記録されている作成者のオペレーター公開鍵の下で検証を通る。

(B2) b.parents 中のすべての親識別子が DAG_v に存在し、参照された各親は b.round より厳密に小さいラウンド番号を持つ。

(B3) b.parents 中の最初の親は、v が知る限りの b.author が作成した最新のブロックである(first-self-parent 規則)。

(B4) b.parents 中のラウンド (b.round − 1) の親作成者集合が、$S_{b.epoch}$ の下でステーク定足数(§2.5)を構成する。

(B5) b.payload のシリアライズ合計サイズが MAX_BLOCK_BYTES を超えない。

規則 B1〜B4 は、B4 における個数加重の「2f + 1」をステーク加重定足数で置き換えた Mysticeti の有効性規則そのものである。規則 B5 は局所的な資源上限であり、すべての正直なバリデーターが同一のパラメータで強制する。したがって BFT 抽象の一部でなくとも、この規則はコンセンサスから可視である。

10.3. コミット規則

MISAKA は Mysticeti の 3 段階コミット規則 [2, §III-B] をそのまま用いる。ここではそのステーク加重版を概略する。提案者スロットとは、バリデーターとラウンドの組 (v, r) である。MISAKA の既定設定では、ラウンドあたり 2 つの提案者スロットがあり、エポック番号をシードとするラウンドロビン置換で決定的に順序付けられている。各スロットは初期状態 undecided にある。

バリデーター v がラウンド r に提案したブロック b について、次のように定義する。

(D1) ラウンド r + 1 のブロック b' が b をサポートするとは、b' から親参照をたどる深さ優先探索で、ラウンド r における v の最初のブロックとして b に出会うことをいう。

(D2) ブロック b が認証済み(certified)であるとは、ラウンド r + 1 のブロックのステーク定足数がサポートを与えることをいう。

(D3) ラウンド r + 2 のブロック c が b に対する証明書(certificate)であるとは、その因果履歴がサポートブロックのステーク定足数を含むことをいう。

コミット規則は次のとおりである。

直接決定規則。 ラウンド r + 2 のブロックのステーク定足数がスロット内のいずれかの提案に対する証明書であれば、スロットは to-commit とマークされる。スロット内のすべての提案について、ラウンド r + 1 のブロックのステーク定足数がサポートしない場合、to-skip とマークされる。いずれでもなければスロットは undecided のままである。

間接決定規則。 直接規則でスロット s が決定されない場合、バリデーターは提案者スロット順序の前方を検索し、ラウンド r' > r + 2 にあってすでに to-commit または undecided となっている最初のスロット s' を見つける。s' が undecided であれば、s も undecided のままである。s' が アンカーブロック a を持つ to-commit であれば、a の因果履歴が s のいずれかの提案に対する証明書を含む場合に限り s を to-commit とマークする。そうでなければ to-skip とマークする。

コミット系列。 全スロットを処理した後、バリデーターは提案者スロット順序を反復し、各 to-commit ブロックを L_v に追記し、各 to-skip スロットをスキップし、最初のまだ undecided であるスロットに到達したところで停止する。

良好なネットワーク条件下では、直接規則は 3 回のメッセージ遅延(propose → support → certify)ですべてのスロットを決定し、Martin と Alvisi のビザンチン下界 [10] に一致する。クラッシュ故障または非同期下では、間接規則は高々追加数ラウンドだけ決定を遅らせる。形式的ライブネス議論は §15 で与える。

10.4. 線形化

リーダーブロック b がコミットされたら、その因果履歴は決定的なトランザクション順序に線形化されなければならない。MISAKA は Mysticeti と同じ線形化を用いる。すなわち、b を根とする部分 DAG であり、かつこれまでのどのリーダーによってもコミットされていないブロックから成る部分木の後行順走査である。兄弟間の順位は、VS_v におけるバリデーター添字で決定する。各ブロック内のトランザクションは payload に現れる順に取られる。走査の後半で再び出会う重複トランザクションはエラーなしで捨てられる。

この線形化は、コミット済み DAG とエポックのバリデーター集合順序の純粋な関数であり、どちらも §8.2 および §8.4 により正直なバリデーター間で一致する。したがって、すべての正直なバリデーターについてビット単位で同一のトランザクション系列を生成する。これは手続き 5 でビット単位の state_root 一致が成立するための前提である。

10.5. ライブネスタイムアウト

Mysticeti と同様、MISAKA は各ラウンドの主要提案者に対して単一のライブネスタイムアウトに依存する。ラウンド r − 1 のブロックのステーク定足数を受理した後、正直なバリデーターは、ラウンド r の主要提案者のブロックが届くのを LEADER_TIMEOUT = 3Δ まで待ってから、自分のラウンド r ブロックを提案する。テストネットの既定設定値は 1 秒である。これは GST 以降、正直な主要提案者のブロックがすべての他の正直なバリデーターによって親として参照されることを保証し、その結果として認証され最終的に直接規則によってコミットされることが保証される。

11. エポック変更と再構成

エポックとは、委員会 C_e が固定されている最大限のラウンド連続区間である。エポック境界では委員会が変更され得る。バリデーターは加わり、去り、またはステーク量を更新され得る。本節ではエポック遷移がどのように検出され、実行され、永続化されて、境界をまたいでチェーンがライブかつ安全に保たれるかを規定する。

11.1. エポック長

エポックは設定可能な壁時計期間 EPOCH_DURATION_SECS の後に終了する。本番設定では 30 日に設定されている。正確な境界は、エポック期限以降に最初にコミットされるリーダーである。この規則はコミット済みタイムスタンプ(§10.1)のみに依存し、バリデーターの壁時計には依存しないため、すべての正直なバリデーターが境界について決定的に合意する。

11.2. バリデーター選出

各エポック境界では、次エポックのアクティブ集合 V_e がオンチェーンのバリデーターレジストリから再計算される。選出規則は次のとおりである。

(R1) レジストリを active_stake ⊠ MIN_STAKE_ACTIVE (= 10^7 MISAKA)かつ score ⊠ MIN_ACTIVE_SCORE (= 0.95)の候補に絞り込む。いずれかの閾値を満たさない候補は除外される。

(R2) 各候補の報酬重みを次で計算する。

$$w_i = \mathrm{isqrt}(\mathrm{stake}_i) \cdot \mathrm{score}_i$$

ここで isqrt は §3.4 の整数平方根(決定的、分岐なし、U128)である。

(R3) 候補を w_i の降順に並べ、同順位はバリデーター ID の昇順で破る。上位 21 名を取って V_{e+1} を構成する。

平方根重み付けは §1.1 の貢献 C4 である。100 倍のステークを持つ候補は 10 倍の報酬重みしか得られず、ステーク集中に対する限界リターンが大きく減じられる一方、ステークに対する厳密な単調性は保たれる。

スコアはエポックごとの性能観測値の平滑化指数加重移動平均である。

$$\mathrm{score}_t = \alpha \cdot \mathrm{score}_{t-1} + (1 - \alpha) \cdot \mathrm{obs}_t$$

参照実装では $\alpha = 0.9$ である。エポックごとの観測値 obs_t は、当該エポックのリーダーラウンドにおけるバリデーターのブロックが成功裏にコミットされた割合であり、4 桁の 10 進固定小数点演算で計算される。MISAKA はスラッシングを実装しない。MIN_ACTIVE_SCORE を下回ったバリデーターは次のエポック境界でアクティブ委員会から外されるが、ステークは保持し、スコアが回復すればアクティブ集合に再参加できる。

11.3. 投票重みと報酬重み

重み関数の 2 つの用途を区別することが重要である。コンセンサスエンジン中の投票重みは、バリデーターの active_stake を最小単位でそのまま用いる。これが §2.5 のステーク定足数述語に入る値であり、バリデーター集合が定足数を形成するかを決定する。一方、報酬重みは isqrt(active_stake) × score であり、(a) 上位 21 名のアクティブバリデーターの選出、(b) エポックごとの報酬プールを分配する際のみに用いる。したがって平方根関数は収入と委員会メンバーシップに影響するが、安全性には影響しない。安全性は完全に線形ステーク定足数によって支配される。これは BFT 的に正しい量である。

11.4. エポック遷移手続き

エポック遷移自体が決定的な手続きであり、各正直なバリデーターがコミット済みリーダー系列の同一の点で実行する。

手続き 6: epoch_transition(e)

```
1. let registry :=
validator_registry_state(UTXO_v)
2. let new_committee :=
select_committee(registry) # §11.2
3. let next_epoch := e + 1
4. VS_v.insert(next_epoch, new_committee)
5. let final_root := UTXO_v.muhash()
6. WAL_v.append(EpochClosed {
    epoch := e,
    final_state_root := final_root,
    next_committee := hash(new_committee),
  })
7. self.epoch := next_epoch
8. dag_engine.reset_for_epoch(next_epoch,
new_committee)
9. retire_old_dag_blocks(retention_rounds)
```

ステップ 1 のバリデーターレジストリ自体が UTXO 状態の一部である。ステークの bond および unbond 操作は通常の MISAKA トランザクションであり、そのエフェクトが指定されたレジストリ出力を更新する。つまり select_committee への入力は、コミット済みのトランザクション順序で完全に決定される。したがってすべての正直なバリデーターは、バリデーター間の明示的な調整なしに同一の new_committee 値を計算する。

11.5. クロスエポック安全性

MISAKA は純粋なコンセンサスチェーンである(コンセンサスなしの高速パスを持たない)ため、エポック境界で要求される安全性は [2, §V-B] で議論される 2 ケースのうち単純な方である。すなわち、エポック e でコミットされたすべてのトランザクションはエポック e + 1 でもコミット済みで留まらなければならない。これは epoch_transition の構成から直ちに従う。すなわち、特定のリーダーがコミットされて実行された後に EpochClosed レコードを WAL に追記してエポックを閉じ、次のエポックはまさにその状態から開始する。ファストパス確定とコンセンサス確定の競合を管理する必要がない。EpochClosed レコードに保持される final_state_root フィールドにより、新エポックに参加するバリデーターは、エポック全体の履歴を再実行せずに正しい状態から開始したことを検証できる。

12. ライトクライアント

ライトクライアントとは、チェーン全体を保存したり実行したりすることなく、MISAKA 台帳の特定の事実ートランザクションの包含、UTXO の値、アドレスの内容ーを検証したい参加者である。ライトクライアントはモバイルウォレット、ブラウザ拡張、取引所の引き出し検証器、および完全バリデーターの帯域・ストレージコストを負担できない任意のアプリケーションにとって不可欠である。本節では MISAKA の想定ライトクライアントプロトコルを規定する。v1 参照実装はまだライトクライアントを同梱していない。ここで記述する設計は v1.1 の目標である。

12.1. ヘッダーチェーン

ライトクライアントはヘッダーチェーンを追跡する。すなわち、コミット済みリーダーブロックヘッダーの系列であり、ヘッダーとは §10.1 の表 7 のフィールド集合から payload を除外したものである。ヘッダーは概ね 4 KB(ML-DSA-65 署名の 3.3 KB に加えて親ハッシュ、ルート、メタデータ)であり、完全なブロックは数百 KB に達し得るのと対照的である。ライトクライアントはヘッダーのみをダウンロードし、加えて関心のあるヘッダーの本体に対する選択的なマークル証明のみを取得する。

12.2. 信頼アンカー: バリデーター集合

ライトクライアントはジェネシス VS_0 の信頼できるスナップショットから開始する。これは初期のバリデーター識別子と ML-DSA-65 公開鍵の集合である。このスナップショットは帯域外で配布される(公式リリースの tar ボール、署名付き更新チャネル、複数の独立ソースの比較など)。このアンカーを所与として、ライトクライアントは §11.4 の EpochClosed レコードのチェーンを辿る。各レコードは(作成者署名を検証できるリーダーブロックにコミットされることで間接的に)署名されており、次エポックの委員会のハッシュを含む。したがってクライアントは信頼を 1 エポックずつ前に拡張し、過去のエポックを再検証する必要はない。

これは構造的には Cosmos ライトクライアントの「信頼済みチェックポイント」機構、および Ethereum ビーコンチェーンの「同期委員会」機構と同一である。耐量子特殊化は署名方式のみに局所化されている。基礎となる署名が Ed25519、BLS、ML-DSA-65 のいずれであろうと、同じ再帰的信頼構造が機能する。

12.3. 定足数証明書の検証

ライトクライアントは、リーダーブロックがコミットされたことを、現エポックの委員会のステーク定足数からの ML-DSA-65 個別署名を伴うことを確認することで検証する。具体的には、トランザクション tx の包含証明は次の要素から成る。

(P1) effects_root を含むコミット済みリーダーヘッダー h。

(P2) tx のエフェクトダイジェストの h.effects_root に対するマークル包含証明。

(P3) ML-DSA-65 署名を保持する後続ブロックヘッダー集合で、作成者の合計が VS[h.epoch] の下で h へのステーク定足数を形成するもの(§10.3 の Mysticeti コミット規則により、認証 1 ラウンドとサポート 1 ラウンドで十分)。

ライトクライアントは (P3) を、各 ML-DSA-65 署名を VS 中の対応するオペレーター公開鍵に対して検証し、信頼済み委員会からの対応するステーク値を合計し、合計が $(2 / 3) \cdot T_e$ を超えることを確認することで検証する。検証コストは署名チェックに支配される。21 バリデーターで定足数に達するのに約 14 個の署名が必要であるから、最悪の場合 14 回の ML-DSA-65 検証となり、現代のモバイル CPU でおよそ 2 ms である。

12.4. 署名集約の不在

BLS ベースのチェーンは、バリデーター署名の定足数を単一の 96 バイト集約に圧縮するために署名集約を用いる。これによりヘッダーサイズと検証コストの両方が削減される。本ドラフト時点で、ML-DSA-65 に対する同等の集約方式は存在しない。BLS 集約を成り立たせる構造的証明は格子方式には転用できず、研究文献中の耐量子集約候補方式はいまだ標準化されていないか、法外に大きな公開パラメータを必要とする。MISAKA は定足数証明書あたりバリデーターあたり 1 つの独立した ML-DSA-65 署名を保持する。このオーバーヘッドは耐量子安全性のコストとして受け入れ、その定量的影響は §14 で議論する。

13. トークノミクス

MISAKA の金融政策は 4 つのパラメータによって支配される。固定最大供給量、恒久下限を持つ逓減インフレーション曲線、手数料率優先のトランザクション手数料モデル、およびスナップショット永続性のあるオンチェーン供給量追跡である。本節では各々を順に規定する。

13.1. 最大供給量と単位

固定最大供給量は MAX_SUPPLY = 10^{10} MISAKA である。1 MISAKA は 10^{9} 最小単位に等しいため、MAX_SUPPLY は 10^{19} 最小単位となる。制約

$$\mathrm{Supply}_v.\mathrm{total} \le \mathrm{MAX_SUPPLY}$$

は §8.5 で規定されるとおり、すべての発行操作で強制される。この制約に違反する発行の試みはエラーを返し、当該ブロックはすべての正直なバリデーターによって拒否される。

13.2. 発行曲線

MISAKA は区分的線形な整数演算インフレーション曲線を用いる。チェーン年 y の基準点単位(basis points)での年間インフレーション率を r(y) と表すと、次が成り立つ。

$$r(y) = \max(R_0 - D \cdot y,\ R_{floor})$$

ここで R_0 = 300 bps (3.00%)、D = 50 bps(0.50%/年)、R_{floor} = 100 bps(1.00%)である。下限に達するのは次の時点である。

$$y_{floor} = (R_0 - R_{floor}) / D = (300 - 100) / 50 = 4,$$

すなわちチェーンの 4 年目である。4 年目以降はインフレーション率は 1.00% のまま恒久的に推移する。

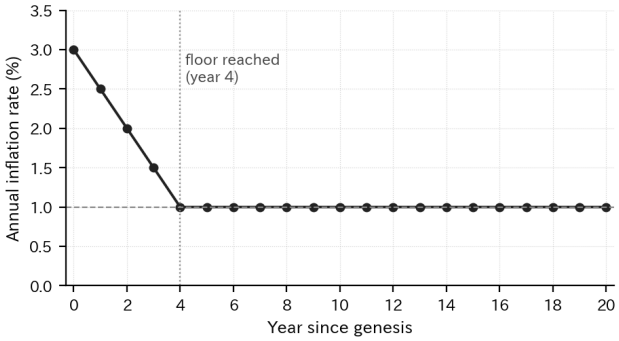

図 1: MISAKA の年間インフレーション曲線。率は 4 年間で 3.00% から 1.00% まで逓減し、以後は下限のまま推移する。

エポックごとの発行量は浮動小数点を用いない整数演算で計算される。

$$\mathrm{emission}(y) = \lfloor (S_y \cdot r(y)) / (10^4 \cdot E) \rfloor$$

ここで S_y は年 y の開始時点での総供給量、r(y) は基準点単位の年率、E は 1 年あたりのエポック数である。除算の順序—基準点を先に、次にエポック数で除算—は u128 中間値の精度損失を最小化するために選ばれている。実装では供給量 × 率の積に checked_mul を用い、オーバーフロー時には除算先行の経路にフォールバックすることで、極端な供給量の下でも決定的挙動を保証する。

13.3. コインベース配分

コミット済みの各リーダーブロックは、コミット済み部分 DAG と供給量追跡から決定的に構成される、ちょうど 1 つのコインベーストランザクションを保持する。コインベースは次の 2 つの出力を生成する。

提案者出力: 当該リーダーのスロットに対するエポックごとのインフレーション発行量と、リーダーの線形化された因果履歴に由来する手数料のうち提案者の取り分。

トレジャリー出力: 手数料のうち設定可能な割合が、オンチェーンパラメータで制御される指定されたトレジャリーアドレスへ送られる。トレジャリー割合はテストネットの既定では 0 であり、将来のエコシステム資金調達のために予約される。

どちらの出力も、すべてのバリデーターが利用可能なデータ(リーダーヘッダー、線形化順序、オンチェーンパラメータ集合)から決定的に導出されるため、2 名の正直なバリデーターはそれぞれ独立にビット単位で同一のコインベーストランザクションを生成する。

13.4. 手数料モデル

MISAKA は Bitcoin や Kaspa の流儀にならった手数料率優先のメンプールモデルを用いる。各トランザクションは fee フィールドを明示的に保持し、その値は入力値と出力値の差に等しい。バリデーターのメンプールは手数料率を fee / size として計算し、保留中のトランザクションをその降順で順序付ける。EIP-1559 の意味でのベース手数料機構、手数料焼却、ブロックあたりのガス目標値のいずれも存在しない。このミニマリズムの動機は二つある。第一に、EIP-1559 のベース手数料計算はブロックごとに合意される新たなコンセンサス可視状態を導入するが、これは DAG ベースのコミット規則と非自明に相互作用する。第二に、スループットが現在の需要を大きく超えるチェーン(§14)では、動的ベース手数料機構はベース手数料を最小値に固定するだけであり、何の利益も提供しない。

手数料は §13.3 のコインベース出力を通じて提案者および(任意で)トレジャリーに支払われる。MISAKA は定常状態では手数料を焼却しない。したがってバリデーターの長期経済インセンティブは、1% の恒久インフレーション下限およびトランザクション手数料収益の合計であり、普及が進むにつれて後者が支配的になることが期待される。

13.5. ステーキングとボンディング

ステーク操作は、出力がステークスクリプトにコミットされる通常の MISAKA トランザクションである。このロックはステーク保有者の ML-DSA-65 署名と、アンボンディング期間の経過時間との両方を要求する。ボンディングは価値を $\text{Supply}_v.\text{circulating}$ から $\text{Supply}_v.\text{staked}$ へ移動する。アンボンディングは UNBOND_PERIOD_EPOCHS の待機期間を開始し、その後価値は元に戻る。待機期間中、ステークはバリデーターの投票重みにも報酬重みにも算入されないが、スラッシング対象でもない(MISAKA はスラッシングを行わない。§11.2 を参照)。

14. スケーリングと遅延

本節では MISAKA のスループットおよび遅延の上界を解析的に扱う。広域ネットワーク上の実測はテストネット展開に基づく付随論文に意図的に先送りする。以下に示す数値はメッセージと署名のサイズのみから導出され、MISAKA の耐量子設計が基盤となる Mysticeti に対して課す定数倍の影響を表している。

14.1. ブロックレベルのオーバーヘッド

§3.1 より、ML-DSA-65 署名は 3309 バイト、公開鍵は 1952 バイトである。MISAKA のブロック(表 7)はちょうど 1 つの署名を持ち、親は 32 バイトハッシュで参照される。21 バリデーター委員会で、定足数におよそ 14 個の親が必要とされる場合、ブロックあたりの固定暗号オーバーヘッドは次のとおりである。

$$3309\ (\text{sig}) + 14\times 32\ (\text{parents}) + 64\ (\text{roots}) \approx 3.8\ \text{KB}.$$

これはトランザクションペイロードを問わないブロックのワイヤーサイズの下限である。比較として、Ed25519 署名の Mysticeti ブロックのワイヤーサイズ下限は概ね 0.6 KB である。MISAKA のオーバーヘッドはブロックあたりおよそ 3.2 KB 多く、単一の ML-DSA-65 署名に支配される。

14.2. ネットワークレベルのオーバーヘッド

各バリデーターはラウンドあたり 1 つのブロックを $n-1$ の他のバリデーターすべてにブロードキャストする。$n = 21$ および 2 秒ラウンドでは、各バリデーターは自ブロックを 1 ラウンドあたり 20 部送信し、他のブロックを 20 部受信する。署名だけの通信量は次のとおりである。

$$20 \cdot 3309\ \text{B} / 2\ \text{s} \approx 33\ \text{KB/s} \approx 264\ \text{kbps}.$$

受信方向と合わせて倍にすると、バリデーターあたりおよそ 528 kbps の純粋な署名通信量となり、バリデーターハードウェアで一般的な 1 Gbps リンクに対しては無視できる。対応する Ed25519 の値は概ね 10 kbps であり、MISAKA の署名通信オーバーヘッドは絶対値で概ね 50 倍高いが、現実的な帯域ボトルネックを大きく下回る。

14.3. 検証 CPU

ML-DSA-65 署名検証は、FIPS 204 参照実装 [3] によれば一般的な x86 ハードウェア上で Ed25519 検証のおよそ 1.4 倍のコストである。21 バリデーター、2 秒ラウンドでは、各バリデーターは 2 秒ごとに 20 個の受信ブロック署名と、入力ごとのトランザクション署名を検証する必要がある。ブロックレベル検証については毎秒およそ 10 検証となり、これは単一 CPU コアの範囲に十分収まる。トランザクションごとの署名検証はスループットに対して線形であり、コア間で自明に並列化可能である。

14.4. 遅延

§10.3 の Mysticeti コミット規則は、良好なネットワーク条件下ではリーダーブロックを 3 メッセージ遅延でコミットし、Martin-Alvisi のビザンチン下界 [10] に一致する。MISAKA はこの遅延を継承する。ML-DSA-65 への置き換えは署名と検証の操作ごとに小さな定数を追加するが、ラウンド構造は変えない。したがって MISAKA の広域コミット遅延は、§14.2 および §14.3 の定数倍の範囲内で、[2, 表 I] で報告される Mysticeti ベースライン(137 バリデーターで P50 650 ms / P95 975 ms)に追随することが期待される。実測による検証は今後のテストネット計測の対象である。

14.5. スループット

Mysticeti 型 DAG におけるスループットは、(i) ラウンド内で提案可能なバリデーター数、(ii) ブロックあたりの最大ペイロードサイズ、(iii) ラウンドレートの積で上界される。21 バリデーターが 2 秒ラウンドごとに 1 回提案し、最大ブロックペイロードがおよそ 1 MB であれば、生のワイヤースループットは 10 MB/s 程度であり、典型的な UTXO トランザクションサイズで毎秒数万トランザクションに相当する。これは現在のブロックチェーン需要を数桁上回っており、当面は律速となる制約ではない。

15. 安全性解析

本節では MISAKA の安全性、ライブネス、耐量子性を論じる。コンセンサスエンジンの安全性およびライブネスは、ステーク加重定足数(§2.5)と ML-DSA-65 署名(§3.1)への置き換えを施して Mysticeti [2] から継承する。4 つの BAB 性質を定理として再掲し、証明を対応する Mysticeti の補題への参照で概略する。

15.1. 安全性

定理 1(合意)。 正直なバリデーター v_i および v_j がトランザクション tx および tx′ を配送するならば、v_i の配送系列は v_j の配送系列の接頭辞であるか、またはその逆が成り立つ。

証明概要。 [2] の補題 2〜7 を、§2.5 のステーク加重定足数述語で個数加重の「2f + 1」述語を置き換えて適用する。Mysticeti の証明は定足数交叉のみを使用し、それはステーク加重定式化で保たれる。任意の 2 つのステーク定足数は、合計ステークが T_e / 3 を超えるバリデーター集合で交わり、§2.2 の境界の下で少なくとも 1 名の正直なバリデーターを含む。ML-DSA-65 による Ed25519 の置き換えは手続き 2 のステップ 5 で用いられる verify_signature プリミティブにのみ影響し、証明の構造を変えない。ML-DSA-65 は MLWE/MSIS の下で EUF-CMA 安全であると仮定する。残りのステップ—コミット済みリーダーの線形化(§10.4)と実行(手続き 5)—は、コミット済みリーダーログ L_v の決定的関数であり、これは [2] の補題 7 により正直なバリデーター間で一致する。よって配送系列は一致する。☒

定理 2(整合性)。 各正直なバリデーターは各トランザクションを高々 1 回配送し、配送するのはいずれかのバリデーターが事前にブロードキャストした場合に限る。

証明概要。 §10.4 の線形化は、リーダーの因果履歴の後行順走査中に複数回出会うトランザクションを重複排除する。トランザクションが $UTXO_v$ に入るのは手続き 5 のステップ 2c のみであり、これは該当するトランザクションがコミット済み部分 DAG のいずれかのブロック b のペイロードに含まれていたことを要求する。そのブロックは手続き 2 によって DAG_v に受理されており、手続き 2 は b 上の作成者署名を検証するため、いずれかのバリデーターが当該トランザクションをブロードキャストしたことになる。☒

定理 3(有効性)。 正直なバリデーターが well-formed なトランザクション tx をブロードキャストしたならば、ネットワークが同期期間にあり、tx が実行中の状態に対して有効であり続ける限り、すべての正直なバリデーターがいずれ tx を配送する。

証明概要。 P2P トランスポートのゴシップ性により、tx はブロードキャスト後有限時間内にすべての正直なバリデーターのメンプールに到達する。ステーク加重定足数に適用された [2] の定理 5 のライブネス議論により、正直な提案者はいずれ tx を含むブロックを提案し、それは Mysticeti コミット規則によりコミットされる。定理 1 により、このコミットはすべての正直なバリデーターが観測し、したがって手続き 5 によって tx を配送する。☒

定理 4(全順序)。 2 名の正直なバリデーターがトランザクション tx および tx′ を配送するならば、両者は同一の相対順序で配送する。

証明概要。 コミット済みリーダーログ L_v は定理 1 により正直なバリデーター間で一致し、§10.4 の線形

化関数は決定的である。よって配送可能順序はすべての正直なバリデーター上で同一である。□

15.2. 耐量子性

定理 5(耐量子偽造不可能性)。
多項式時間の量子敵性者に対する ML-DSA-65 の EUF-CMA 安全性の下で、量子で制限された敵性者は、秘密鍵にアクセスすることなく正直なバリデーターを名乗る有効な署名を生成することができない。

証明概要。 §3.1 の EUF-CMA 仮定から直接従う。手続き 2 のステップ 5 で受理される任意の署名は、VS_v に記録されたバリデーターのオペレーター公開鍵の下で検証を通らねばならない。そのような署名を秘密鍵なしで生成することは EUF-CMA に矛盾する。□

定理 6(P2P 通信の耐量子機密性)。
多項式時間の量子敵性者に対する ML-KEM-768 の IND-CCA2 安全性、および古典敵性者に対する ChaCha20-Poly1305 の AEAD 安全性の下で、完全な P2P 通信を記録し後に CRQC を獲得したネットワーク観測者は、正直なバリデーター間の任意のセッションの平文を復元できない。

証明概要。 ML-KEM-768 共有秘密から HKDF-SHA3-256 によって導出されるセッション鍵は、記録された暗号文から IND-CCA2 の下で復元できない。それは将来の量子敵性者であっても同様である。なぜなら量子敵性者に対する IND-CCA2 こそが FIPS 203 の仮定だからである。セッション鍵がなければ、ChaCha20-Poly1305 下の AEAD 暗号文は、標準対称鍵モデルにおいて任意の古典あるいは量子敵性者から見てランダムと区別不能である。□

15.3. 供給量のスナップショット永続性

定理 7(再起動下での上限強制)。 バリデーター v に対する任意のクラッシュ・再起動系列の後、永続化された $Supply_v$.total は、最後に完全に実行されたリーダーまでに L_v にコミットされた累積発行量にちょうど等しい。特に、再起動によって v が既にコミット済みの報酬を再発行することはない。

証明概要。 手続き 5 のステップ 5 は $Supply_v$.mint(·) を、ステップ 8 で BlockExecuted を WAL_v に追記する前に呼び出し、発行は UTXO 書き込みと同一の原子的ストレージトランザクションの一部である。クラッシュリカバリ時、WAL が再生され、供給量追跡は最後の BlockExecuted エントリによって記録された状態に正確に復元される。§8.5 の起動プロトコルは追跡を読み込めない場合に起動を拒否するため、復旧したバリデーターが追跡をジェネシス値から再初期化してしまう故障モードが排除される。□

16. 関連研究

DAG ベースの BFT コンセンサス。 MISAKA のコンセンサスエンジンは、構造化 DAG コンセンサスの系譜ーHashGraph [11]、DagRider [12]、Narwhal/Tusk [5]、Bullshark [6]、Mysticeti [2]ーに属する。MISAKA が継承する Mysticeti の決定的革新は、証明なし DAG ブロックの使用である。別個のコンセンサスインスタンスで各ブロックを認証する代わりに、認証は DAG 構造の中に暗黙に含まれ、コミット規則がそれを都度解釈する。これによりコミット遅延がおよそ 6 メッセージ遅延(Bullshark)から 3(Mysticeti、MISAKA)へ低減され、Martin と Alvisi のビザンチン下界 [10] に一致する。

耐量子ブロックチェーン。 本番ブロックチェーンにおける耐量子暗号は、ジェネシスからの性質としてではなく、将来の移行パスとして枠づけされることが最も多い。いくつかの研究提案は、移行的措置として楕円曲線成分を保持するハイブリッド方式を議論している。§1 で論じたように、そのようなハイブリッドはハーベストナウ・デクリプトレイター攻撃に対して依然として脆弱なため、耐量子前方秘匿性の目標を損なう。「ジェネシスから耐量子」の立場を取る最も近い構造的同類は、XMSS ハッシュベース署名を使用する Quantum Resistant Ledger(QRL)である。ただし QRL のコンセンサスはプルーフオブワークで、データモデルはアカウントベースであるため、MISAKA との比較は直接的ではなく構造的なものである。筆者の知る限り、MISAKA は、コンセンサス、トランスポート、およびユーザ署名方式のすべてを格子暗号のみで構築した Layer 1 ブロックチェーンの完全仕様としては初めてのものである。

UTXO チェーン。 透明 UTXO モデルは Bitcoin [8] に由来し、Litecoin、Monero(プライバシー拡張付き)、Cardano(拡張 UTXO)、Kaspa に採用されている。MISAKA のスクリプトエンジンは、実装されているオペコードの部分集合において Kaspa と互換であり、ML-DSA-65 署名オペコード(§4.4)が追加されている。初期の MISAKA プロトタイプにはペアリングベース暗号を用いた環署名およびステルスアドレスのプライバシー機能が含まれていたが、ペアリング自体が量子に対して脆弱であり、本チェーンの脅威モデルと両立しないことが内部レビューで判明した後に削除された。

DPoS とステーク重み付け。 21 スロットのアクティブ委員会は EOS および Tron の系譜から着想を得ている。平方根重み付け規則は意図的な逸脱である。EOS も Tron もステークの劣線形関数を用いていない。二次投票 [13] は平方根選択を、大規模保有者の限界影響力を低減する手段として動機づけ、Cardano の Ouroboros プロトコルファミリ [11] はスラッシングを行わない選択を、スラッシングが中央集権的な staking-as-a-service プロバイダに過大なインセンティブを生み出すという理由で動機づけた。

17. 結論

本文書では MISAKA Network の仕様を定義した。これはコンセンサスエンジン、P2P トランスポート、ユーザレベル署名方式のすべてを格子暗号のみで構築し、コンセンサスクリティカルパスから楕円曲線暗号を明示的に排除した Layer 1 ブロックチェーンである。MISAKA は、ML-DSA-65 で認証される透明 UTXO 台帳、未使用集合に対する SHA3-256 MuHash インクリメンタルコミットメント、ML-KEM-768 暗号化 P2P トランスポート、Mysticeti から移植した Narwhal/Bullshark 証明なし DAG コンセンサスエンジン、平方根報酬重み付けの 21 スロット DPoS 委員会、スナップショット永続性のある供給量追跡を組み合わせる。本設計は耐量子脅威モデルを将来の移行項目ではなくジェネシスからの性質として取り扱い、参照実装は現在公開テストネット上で稼働中である。

未解決問題。 今後の課題として複数の問いが残る。第一に、MISAKA プロトコル全体ーコンセンサスコアのみでなくーの MLWE および MSIS 困難性仮定への完全な形式的還元は、安全性議論を強化するであろう。第二に、§14 の解析的上界を検証するため、21 バリデーターの地理分散テストベッドにおける実測評価が必要である。第三に、§12 の設計に合致する本番運用可能なライトクライアントが実装され監査されなければならない。第四に、暗号パラメータ変更を経てもステークの永続性を保ち得る形で、バリデーター集合パラメータおよび発行曲線のオンチェーンガバナンスを規定する必要がある。

参考文献

[1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in Proc. 35th FOCS, 1994.

[2] K. Babel, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, A. Koshy, A. Sonnino, and M. Tian, “Mysticeti: Reaching the latency limits with uncertified DAGs,” in NDSS Symposium, 2025.

[3] National Institute of Standards and Technology, Module-Lattice-Based Digital Signature Standard, NIST FIPS 204, 2024.

[4] National Institute of Standards and Technology, Module-Lattice-Based Key-Encapsulation Mechanism Standard, NIST FIPS 203, 2024.

[5] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus,” in EuroSys, 2022.

[6] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: DAG BFT protocols made practical,” in ACM CCS, 2022.

[7] National Institute of Standards and Technology, SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions, NIST FIPS 202, 2015.

[8] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.

[9] S. Blackshear, A. Chursin, G. Danezis et al., “Sui Lutris: A blockchain combining broadcast and consensus,” arXiv preprint, 2023.

[10] J. Martin and L. Alvisi, “Fast Byzantine consensus,” IEEE TDSC, vol. 3, no. 3, 2006.

[11] A. Kiayias, A. Russell, B. David, and R. Oliynykov, “Ouroboros: A provably secure proof-of-stake blockchain protocol,” in CRYPTO, 2017.

[12] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is DAG,” in PODC, 2021.

[13] E. G. Weyl, “Quadratic voting and the public good,” Public Choice, vol. 172, 2017.

[14] L. Baird, “The Swirlds Hashgraph consensus algorithm: Fair, fast, Byzantine fault tolerance,” Swirlds Tech Report, 2016.

[15] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in OSDI, 1999.

[16] National Institute of Standards and Technology, SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash, NIST SP 800-185, 2016.

付録 A. スクリプトオペコード一覧

表 A.1 は MISAKA v1 スクリプトエンジンで実装されるオペコードの一覧である。この集合は意図的に最小限に抑えられており、標準的な単一署名ロックと少数のスタック操作プリミティブをサポートし、チューリング完全な制御フローを持たない。

オペコード	Hex	説明
`OP_0`	0x00	空のバイト列をプッシュ。
`OP_DUP`	0x76	スタック先頭要素を複製。
`OP_HASH256`	0xAA	先頭要素をその SHA3-256 ハッシュで置き換え。
`OP_EQUALVERIFY`	0x88	2 要素をポップし、等しくなければ失敗。

OP_CHECKSIG_MLDSA	0xC0	(pk, sig) をポップ。Verify(pk, sighash, sig) が真なら 1、偽なら 0 をプッシュ。
OP_CHECKSIGVERIFY_MLDSA	0xC1	上と同様だが、結果が 0 のときスクリプトを失敗させる。

表 A.1: MISAKA v1 スクリプトオペコード。

付録 B. 定数

定数	値
MAX_SUPPLY	10^{10} MISAKA = 10^{19} 最小単位
DECIMALS	9
MIN_STAKE_ACTIVE	10^{7} MISAKA
MIN_STAKE_BACKUP	10^{6} MISAKA
ACTIVE_SET_SIZE	21
MIN_ACTIVE_SCORE	0.95
EWMA_ALPHA	0.9
INITIAL_INFLATION_BPS	300(3.00%)
ANNUAL_DECAY_BPS	50(0.50%/年)
FLOOR_INFLATION_BPS	100(1.00%)
FLOOR_REACHED_AT_YEAR	4
EPOCH_DURATION_SECS	30 × 24 × 3600(30 日)
LEADER_TIMEOUT	3Δ(≈1 秒、テストネット)
ROUND_TARGET_SECS	2
P2P_PORT (testnet)	6690
RPC_PORT (testnet)	3001
CHAIN_ID (testnet)	2
CHAIN_ID (mainnet)	1

表 B.1: MISAKA v1 プロトコル定数。

付録 C. 謝辞

MISAKA Network は Mysten Labs チームによる Mysticeti 参照実装、および Danezis、Kokoris-Kogias、Sonnino、Spiegelman らとその共同研究者による DAG ベース BFT コンセンサス研究の系譜の直接の上に構築されている。耐量子プリミティブは CRYSTALS-Kyber および CRYSTALS-Dilithium のチームと NIST に由来する。MuHash 構成は Maxwell、Poelstra、Seurin、Wuille らによるものである。適用における誤りはすべて著者自身の責任である。